SAHAYAK Static Sign Collection, Training & Deployment Guide

Copy-and-paste workflow for adding a static two-hand sign, validating the candidate model, promoting it safely, and pushing only the approved model files to GitHub.

Scope	Safe-by-default sequence
Windows local training project plus GitHub deployment repository.	Backup → collect → train candidate → test → promote → sync → Git commit/push → verify the deployed API.

1. Open the correct project

The commands below are for the local training project, not the separate deployment clone. Start PowerShell and paste:

```
Set-Location "C:\Users\user\3D Objects\New folder\SAHAYAK"
.\.venv\Scripts\Activate.ps1
python --version
Test-Path .\collect_data.py
Test-Path .\data\landmarks_2hand.csv
```

The last command should be True if your existing static dataset is present. If virtual-environment activation is blocked, do not change execution policy just for this guide; run the same commands using .\.venv\Scripts\python.exe instead of python.

2. Make a timestamped backup BEFORE collecting

```
$stamp = Get-Date -Format "yyyyMMdd_HHmmss"
$backup = Join-Path $PWD "backups\static_$stamp"
New-Item -ItemType Directory -Path $backup -Force | Out-Null
foreach ($file in @(
  "data\landmarks_2hand.csv",
  "model\sahayak_twohand.joblib",
  "model\labels_twohand.json"
)) {
  if (Test-Path $file) { Copy-Item $file $backup -Force }
}
Write-Host "Backup saved to: $backup"
```

The collection script appends new rows to data/landmarks_2hand.csv. The backup lets you restore the previous data and active static model if needed. The continuous model and databases are not part of this workflow.

3. Collect the new static sign

```
python collect_data.py
```

- When prompted, type the new label exactly as you want it displayed, such as HELLO or THANK_YOU, then press Enter.
- In the webcam window, press S to start saving, press S again to pause; press Q or Esc to finish. On-screen Saved count is the number collected in this run.
- Collect varied examples: hand position, distance, lighting, and natural variation. The script recommends 150-300 samples per sign as a starting target. Keep the meaning and label consistent.
- For a two-hand sign, keep both hands clearly visible. Existing labels remain in the same CSV; do not delete old rows just to add a sign.

Review the real label counts after collection:

```
@'
import csv, collections
from pathlib import Path
p = Path('data/landmarks_2hand.csv')
with p.open(encoding='utf-8-sig', newline='') as f:
    rows = list(csv.DictReader(f))
counts = collections.Counter(row['label'] for row in rows)
print('TOTAL', len(rows))
print(dict(sorted(counts.items())))
'@ | python -
```

4. Train a candidate - this does NOT replace the live model

```
python train_model.py
```

Read the full terminal output. It reports the dataset counts, held-out accuracy, and per-label precision/recall. If the new label is missing, classes are badly imbalanced, or the new sign confuses old labels, collect better samples and train again. Accuracy on a held-out split is only an estimate, not a guarantee of real-world performance.

Run the offline feature tests and check the candidate file:

```
python -m unittest test_sahayak.py
@'
import joblib, json
from pathlib import Path
old = joblib.load('model/sahayak_twohand.joblib')
new = joblib.load('model/sahayak_twohand_candidate.joblib')
labs = json.loads(Path('model/labels_twohand_candidate.json').read_text(encoding='utf-8'))
old_labels = set(map(str, old.classes_))
labels = set(map(str, new.classes_))
assert getattr(new, 'n_features_in_', None) == 126, 'wrong feature count'
assert old_labels.issubset(labels), f'old labels missing: {sorted(old_labels-labels)}'
assert set(labs) == labels, 'candidate labels file mismatch'
print('STATIC CANDIDATE OK:', sorted(labels))
'@ | python -
```

The candidate check confirms a 126-feature model, all prior active labels retained, and matching label metadata. It is not a substitute for testing the new sign with real camera examples after promotion.

5. Promote only after the candidate passes

```
python train_model.py --promote
```

This command retrains and evaluates the candidate, then backs up the active model/labels into model/backups/ and promotes the new model. It is an explicit action; do not run it until you have reviewed the candidate report and tests.

```
python -m unittest test_sahayak.py
python web_app.py
```

Keep the local server running. In a second PowerShell window, check readiness:

```
Invoke-RestMethod "http://127.0.0.1:5000/api/status" | ConvertTo-Json -Depth 8
```

- Confirm model_ready is True and single_sign_labels contains your new label plus all prior labels.
- Open http://127.0.0.1:5000 and test the new sign using the camera, then test representative previously supported signs. Check that the annotation and recognition output move with the hands and are not stuck on one label.
- If testing fails, stop here. Restore the previous model and labels from the timestamped model/backups directory, then retest before uploading anything.

6. Sync the promoted static model into the GitHub deployment tree

Keep the training folder and repository clone separate. In the examples below, $TrainingRoot is your training project and $RepoPath is your local clone of Mrjohanjyotikuli/SAHAYAK-ISL. Change only these two paths if your folders are elsewhere.

```
$TrainingRoot = "C:\Users\user\3D Objects\New folder\SAHAYAK"
$RepoPath = "C:\Users\user\3D Objects\New folder\SAHAYAK-ISL"

# If you have not cloned the deployment repository yet, run this ONCE:
if (-not (Test-Path (Join-Path $RepoPath ".git"))) {
  git clone https://github.com/Mrjohanjyotikuli/SAHAYAK-ISL.git $RepoPath
}

git -C $RepoPath status --short
# Continue only after you have safely dealt with any local changes.
git -C $RepoPath pull --ff-only origin main

# Save a remote backup branch of the current main before the model update.
$backupBranch = "backup/pre-static-" + (Get-Date -Format "yyyyMMdd-HHmmss")
git -C $RepoPath branch $backupBranch main
git -C $RepoPath push origin $backupBranch

# Copy only the promoted model and its labels into the deployment tree.
Set-Location $TrainingRoot
.\sync_model_to_github.ps1 -RepoPath $RepoPath -ModelType Static
```

The helper copies model/sahayak_twohand.joblib to SAHAYAK-Vercel-Deployment/backend/model/ and model/labels_twohand.json to SAHAYAK-Vercel-Deployment/backend/. It does not copy the CSV, candidate files, continuous model, or local databases.

7. Review, commit, and push with Git

```
git -C $RepoPath status --short
git -C $RepoPath diff --stat

# Stage ONLY these static model files:
git -C $RepoPath add `
  SAHAYAK-Vercel-Deployment/backend/model/sahayak_twohand.joblib `
  SAHAYAK-Vercel-Deployment/backend/labels_twohand.json

git -C $RepoPath diff --cached --name-only
# Confirm the staged list contains only the two paths above.
git -C $RepoPath diff --cached --stat
git -C $RepoPath commit -m "Update static sign recognition model"
git -C $RepoPath push origin main
```

Do not stage data/landmarks_2hand.csv, continuous_gesture_db.db, model/backups, .venv, or candidate files. If the staged file list contains anything unexpected, run git -C $RepoPath reset and stage only the two listed paths again.

8. Verify the deployed API and website

```
# Run after the Render deployment has finished.
$status = Invoke-RestMethod "https://sahayak-backend-krh1.onrender.com/api/status"
$status.model_ready
$status.single_sign_labels
```

- Confirm model_ready is True and the new sign appears in single_sign_labels. If not, open the Render service logs and check the last commit/deployment before making another change.
- Open https://sahayak-eight-lac.vercel.app/studio.html and test the new sign and several old signs with the camera.
- A model-only backend update normally needs the backend deployment to finish; do not edit frontend files or manually redeploy Vercel unless the frontend source also changed.

Rollback rule

If the new model regresses, do not force-push. Restore the previous static model and labels from the timestamped model/backups folder, run the same sync helper again, review the two-file diff, commit with a clear rollback message, and push. The remote backup branch created above remains available to compare the repository state.